

01 输入输出篇

目标:看完能写 ACM 模式(牛客)的完整骨架,能在 LeetCode 模式直接写方法。
重点:BufferedReader + StreamTokenizer(最快)、PrintWriter(大输出防 TLE)、throws IOException。
C++ 类比:Scanner ≈ cin, BufferedReader ≈ 自己读 buffer, PrintWriter ≈ 攒一起 cout.flush()。

一、I/O 方式选型(看完直接抄)

方式	速度	易用性	推荐场景
Scanner	慢(内部用正则解析)	高	入门、单组小数据、牛客题面没卡常数
★ BufferedReader + StreamTokenizer	快 5-10 倍	中	ACM 模式首选,数据量大时唯一选择
BufferedReader + StringTokenizer + parseInt	快	中	题解常用,可读性好,比 StreamTokenizer 简单
BufferedReader + split 自己切	中	低	特殊场景(比如字段里带空格需要合并),不推荐

一句话选型:能写 StreamTokenizer 就写 StreamTokenizer,想偷懒用 StringTokenizer,最后才考虑 Scanner。

二、Scanner 家族(先讲坑,再讲 API)

2.1 四个常用方法

方法	返回	跳过空白?	读换行符?	用途
nextInt()	int	是	否	读整数
nextDouble()	double	是	否	读浮点
next()	String	是	否	读一个不带空格的 token
nextLine()	String	否	是	读一整行(到换行符为止)

2.2 换行符陷阱(必踩一次)

nextInt() 不会读走换行符,下一次 nextLine() 会立刻读到空字符串。

```
// 错误:读到的是空串
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();    // 输入:3\n
String line = sc.nextLine(); // line = "",不是 "abc"
System.out.println(line.length()); // 输出 0,一脸懵
```

```
// 正确:多读一次空行 / 改用 BufferedReader
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
sc.nextLine();           // 显式吞掉那个换行符
String line = sc.nextLine(); // 现在能正确读到 "abc"
```

2.3 性能警告

Scanner 内部用正则逐字符匹配,100 万次 nextInt() 大约 3 秒+。牛客如果数据量 10^5 起步,强烈建议直接换 BufferedReader。

三、BufferedReader + StreamTokenizer(最快,ACM 首选)

3.1 三个关键 API

```
BufferedReader br = new BufferedReader(new InputStreamReader(System.in)); // 按行读
StreamTokenizer in = new StreamTokenizer(br);                          // 把每行切成 token
in.nextToken();               // 推进到下一个 token,返回类型(ttype)
in.nval                       // 数字 token 的值(double 类型)
in.sval                       // 字符串 token 的值(String 类型)
```

3.2 完整示例:读 N 个整数到数组

题目:第一行 N,第二行 N 个整数,输出它们的和。

```
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StreamTokenizer in = new StreamTokenizer(br);

        in.nextToken();
        int n = (int) in.nval;    // 强转:nval 是 double

        long sum = 0;           // 用 long 防止溢出
        for (int i = 0; i < n; i++) {
            in.nextToken();
            sum += (long) in.nval;
        }
        System.out.println(sum);
    }
}
```

C++ 对照(感受一下 Java 这套仪式感):



```
#include <iostream>
int main() {
    int n; std::cin >> n;
    long long sum = 0;
    for (int i = 0; i < n; i++) {
        int x; std::cin >> x;
        sum += x;
    }
    std::cout << sum << std::endl;
    return 0;
}
```

3.3 数字类型注意(容易踩的细节)

- `in.nval` 是 `double`, 取整数必须 `(int)` 强转。如果你只想要 `int`, 直接 `(int) in.nval` 就行。
- 如果数据可能超过 `int` (比如 10^9 以上的和), 用 `long`, 先 `(long) in.nval` 再累加。
- `in.nval` 是 `double` 有精度上限: 整数超过 2^{53} (约 9×10^{15}) 会丢精度。大数读入用 `StringTokenizer + Long.parseLong` 代替, 不要用 `StreamTokenizer`。
- `in.sval` 是 `String`, 判等用 `.equals` (参考 06 章 `String` 坑)。

四、BufferedReader + StringTokenizer(题解常用, 可读性好)

4.1 workflow

`readLine()` 读一整行字符串 → `StringTokenizer` 按空格/制表符自动切分 → `parseInt` 逐个取值。

```
String line = br.readLine();           // "1 2 3 4 5"
StringTokenizer st = new StringTokenizer(line);
while (st.hasMoreTokens()) {
    int x = Integer.parseInt(st.nextToken());    // 逐个取出
    // 业务逻辑
}
```

4.2 完整示例: 读一行 N 个整数

题目: 输入 N , 然后一行 N 个整数, 排序后输出。



```

import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());

        int[] a = new int[n];
        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) {
            a[i] = Integer.parseInt(st.nextToken());
        }
        Arrays.sort(a);

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            if (i > 0) sb.append(' ');
            sb.append(a[i]);
        }
        System.out.println(sb);
    }
}

```

小贴士:br.readLine().trim() 加 trim() 是防御性编程,防止行首/行尾有空格导致 parseInt 报错。

五、PrintWriter / BufferedWriter(牛客第一坑)

5.1 为什么不能 System.out.println 多次

System.out 每次 println 都直接刷到系统调用,100 万行输出大约 3 秒+,牛客大数据题直接 TLE。
正确做法:用 StringBuilder 拼好,一次性 PrintWriter.println + flush()。

5.2 完整示例:输出 N 行字符串



```

import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        int n = Integer.parseInt(br.readLine().trim());
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            sb.append(br.readLine()).append("\n");
        }
        out.print(sb);          // 一次性写入
        out.flush();            // 强制刷新,必须加
    }
}

```

C++ 对照:

```

#include <iostream>
#include <string>
int main() {
    int n; std::cin >> n;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // 吞掉 n 后的换行符(std::cin.ignore() 默认只跳 1 字符,不安全)
    std::string line;
    std::string all;
    for (int i = 0; i < n; i++) {
        std::getline(std::cin, line);
        all += line + "\n";
    }
    std::cout << all;
    return 0;
}

```

坑点:PrintWriter 构造时包装 BufferedWriter 是关键,直接 new PrintWriter(System.out) 没加速效果。

5.4 格式化输出 String.format(C++ printf 的对应)

C++ 选手常用的 printf / sprintf,在 Java 对应 String.format(用法和 printf 几乎一样):

```

// C++: printf("%d + %d = %d\n", a, b, a + b);
// Java:
String s = String.format("%d + %d = %d\n", a, b, a + b);
out.println(s);

// 直接输出也可以:out.printf("%d + %d = %d\n", a, b, a + b);
// 区别:printf 是 PrintWriter 的方法,format 是 String 的方法,功能等价

```

典型场景:LeetCode 打印路径、构造特定格式字符串、记录"行号:内容"等。



六、异常声明 throws IOException(Java 特有,无 C++ 对应)

6.1 为什么必须写

Java 把"可能出错的 I/O 操作"标成受检异常(checked exception),`BufferedReader.readLine()`、`br.close()` 都会抛 `IOException`。不写 `throws IOException` 编译报错。

C++ 选手第一次写 Java 会在这一步卡住:C++ 没有这个语法,不写也能跑;Java 不写就编译失败。

6.2 两种处理方式

```
// 方式 1(刷题推荐):main 上 throws,简洁
public static void main(String[] args) throws IOException {
    BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
    // ...
}

// 方式 2(工程向):try-catch 包起来,刷题用不到
public static void main(String[] args) {
    try {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        // ...
    } catch (IOException e) {
        e.printStackTrace();
    }
}
```

速成原则:刷题永远用方式 1(`throws IOException`),省事。本文档不展开 `try-catch-finally`。

七、ACM 模式完整骨架 v2(标准模板,直接背)

```
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        StreamTokenizer in = new StreamTokenizer(br);
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        // ===== 业务逻辑写在这里 =====

        out.flush(); // 必须有,否则可能丢输出
    }
}
```

背下这三行 `import;java.util.*(容器)+ java.io.*(I/O)+ java.util.*` 里的 `Arrays` 也包含在内。再背一个 `throws IOException`,ACM 模式就齐活了。



八、模板代码(7 套,全部可独立跑)

模板 1:单组输入(读两个整数,输出 a+b)

Java(Scanner 版,入门用)

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int a = sc.nextInt();
        int b = sc.nextInt();
        System.out.println(a + b);
    }
}
```

Java(StreamTokenizer 版,推荐)

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));
        in.nextToken(); int a = (int) in.nval;
        in.nextToken(); int b = (int) in.nval;
        System.out.println(a + b);
    }
}
```

C++ 对照

```
#include <iostream>
int main() {
    int a, b;
    std::cin >> a >> b;
    std::cout << a + b << std::endl;
    return 0;
}
```

模板 2a:多组 T 输入(while + 计数器)

题目:第一行 T,接下来 T 行,每行两个整数,输出 a+b。

Java



```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        in.nextToken();
        int T = (int) in.nval;
        StringBuilder sb = new StringBuilder();
        for (int t = 0; t < T; t++) {
            in.nextToken(); int a = (int) in.nval;
            in.nextToken(); int b = (int) in.nval;
            sb.append(a + b).append('\n');
        }
        out.print(sb);
        out.flush();
    }
}
```

C++ 对照

```
#include <iostream>
int main() {
    int T; std::cin >> T;
    while (T--) {
        int a, b; std::cin >> a >> b;
        std::cout << a + b << "\n";
    }
    return 0;
}
```

模板 2b:不告诉 T(读到文件末尾 / 哨兵 0 0 终止)

变体 A:读到 EOF 为止(牛客常见)

Java




```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        StringBuilder sb = new StringBuilder();

        while (in.nextToken() != StreamTokenizer.TT_EOF) {
            int a = (int) in.nval;
            in.nextToken();
            int b = (int) in.nval;
            sb.append(a + b).append('\n');
        }
        out.print(sb);
        out.flush();
    }
}
```

C++ 对照

```
#include <iostream>
int main() {
    int a, b;
    while (std::cin >> a >> b) { // cin 自动判 EOF
        std::cout << a + b << "\n";
    }
    return 0;
}
```

变体 B:哨兵 00 终止(经典套路)

Java

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));
        StringBuilder sb = new StringBuilder();

        while (true) {
            in.nextToken(); int a = (int) in.nval;
            in.nextToken(); int b = (int) in.nval;
            if (a == 0 && b == 0) break;
            sb.append(a + b).append('\n');
        }
        out.print(sb);
        out.flush();
    }
}
```

C++ 对照



```
#include <iostream>
int main() {
    int a, b;
    while (std::cin >> a >> b) {
        if (a == 0 && b == 0) break;
        std::cout << a + b << "\n";
    }
    return 0;
}
```

模板 3:一行 N 个数(两套都要会)

题目:输入 N,然后一行 N 个整数,排序后输出。

Java(StreamTokenizer 版,ACM 最快)

```
import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        StreamTokenizer in = new StreamTokenizer(new BufferedReader(new InputStreamReader(System.in)));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        in.nextToken();
        int n = (int) in.nval;

        int[] a = new int[n];
        for (int i = 0; i < n; i++) {
            in.nextToken();
            a[i] = (int) in.nval;
        }
        Arrays.sort(a);

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            if (i > 0) sb.append(' ');
            sb.append(a[i]);
        }
        out.println(sb);
        out.flush();
    }
}
```

Java(StringTokenizer + parseInt 版,题解常见,更直观)



```

import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        int n = Integer.parseInt(br.readLine().trim());

        int[] a = new int[n];
        StringTokenizer st = new StringTokenizer(br.readLine());
        for (int i = 0; i < n; i++) {
            a[i] = Integer.parseInt(st.nextToken());
        }
        Arrays.sort(a);

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < n; i++) {
            if (i > 0) sb.append(' ');
            sb.append(a[i]);
        }
        System.out.println(sb);
    }
}

```

C++ 对照

```

#include <iostream>
#include <vector>
#include <algorithm>
int main() {
    int n; std::cin >> n;
    std::vector<int> a(n);
    for (int i = 0; i < n; i++) std::cin >> a[i];
    std::sort(a.begin(), a.end());
    for (int i = 0; i < n; i++) {
        if (i > 0) std::cout << ' ';
        std::cout << a[i];
    }
    std::cout << std::endl;
    return 0;
}

```

模板 4:N 行字符串(逆序输出)

题目:输入 N,接下来 N 行字符串,逆序输出。

Java



```

import java.util.*;
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        PrintWriter out = new PrintWriter(new BufferedWriter(new OutputStreamWriter(System.out)));

        int n = Integer.parseInt(br.readLine().trim());
        String[] lines = new String[n];
        for (int i = 0; i < n; i++) {
            lines[i] = br.readLine();
        }

        StringBuilder sb = new StringBuilder();
        for (int i = n - 1; i >= 0; i--) {
            sb.append(lines[i]).append("\n");
        }
        out.print(sb);
        out.flush();
    }
}

```

C++ 对照

```

#include <iostream>
#include <string>
#include <vector>
int main() {
    int n; std::cin >> n;
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n'); // 吞掉 n 后的换行符(std::cin.ignore() 默认只跳 1 字符,不安全)
    std::vector<std::string> lines(n);
    for (int i = 0; i < n; i++) std::getline(std::cin, lines[i]);
    for (int i = n - 1; i >= 0; i--) std::cout << lines[i] << "\n";
    return 0;
}

```

模板 5:LeetCode 链表输入(无显式输入,直接给数组)

LeetCode 206 反转链表,输入 [1,2,3,4,5],输出反转后的链表。

Java



```

import java.util.*;

class ListNode {           // 链表节点类,刷题必备
    int val;
    ListNode next;
    ListNode() {}
    ListNode(int v) { val = v; }
    ListNode(int v, ListNode n) { val = v; next = n; }
}

public class Solution {
    public ListNode reverseList(ListNode head) {
        ListNode prev = null, cur = head;
        while (cur != null) {
            ListNode nxt = cur.next;
            cur.next = prev;
            prev = cur;
            cur = nxt;
        }
        return prev;
    }

    // ===== 以下是本地测试用,LeetCode 提交时删掉 =====
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        ListNode head = buildList(arr);
        Solution sol = new Solution();
        head = sol.reverseList(head);
        printList(head);
    }

    static ListNode buildList(int[] arr) {
        ListNode dummy = new ListNode(0);
        ListNode cur = dummy;
        for (int x : arr) {
            cur.next = new ListNode(x);
            cur = cur.next;
        }
        return dummy.next;
    }

    static void printList(ListNode head) {
        StringBuilder sb = new StringBuilder("");
        for (ListNode p = head; p != null; p = p.next) {
            if (sb.length() > 1) sb.append(",");
            sb.append(p.val);
        }
        sb.append("");
        System.out.println(sb);
    }
}

```

C++ 对照



```

#include <iostream>
#include <vector>
struct ListNode {
    int val;
    ListNode* next;
    ListNode(int v = 0, ListNode* n = nullptr) : val(v), next(n) {}
};

ListNode* reverseList(ListNode* head) {
    ListNode* prev = nullptr;
    for (ListNode* cur = head; cur != nullptr; ) {
        ListNode* nxt = cur->next;
        cur->next = prev;
        prev = cur;
        cur = nxt;
    }
    return prev;
}

// 辅助函数略
int main() {
    std::vector<int> arr = {1,2,3,4,5};
    ListNode dummy(0);
    ListNode* cur = &dummy;
    for (int x : arr) {
        cur->next = new ListNode(x);
        cur = cur->next;
    }
    ListNode* head = reverseList(dummy.next);
    for (ListNode* p = head; p; p = p->next) std::cout << p->val << " ";
    std::cout << std::endl;
}

```

模板 6:LeetCode 二叉树层序输入(LeetCode 标配)

LeetCode 102 层序遍历,输入 [3,9,20,null,null,15,7],输出 [[3],[9,20],[15,7]]。

Java



```

import java.util.*;

class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode() {}
    TreeNode(int v) { val = v; }
    TreeNode(int v, TreeNode l, TreeNode r) { val = v; left = l; right = r; }
}

public class Solution {
    public List<List<Integer>> levelOrder(TreeNode root) {
        List<List<Integer>> res = new ArrayList<>();
        if (root == null) return res;
        ArrayDeque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        while (!q.isEmpty()) {
            int sz = q.size();           // 当前层节点数
            List<Integer> level = new ArrayList<>(sz);
            for (int i = 0; i < sz; i++) {
                TreeNode node = q.poll();
                level.add(node.val);
                if (node.left != null) q.offer(node.left);
                if (node.right != null) q.offer(node.right);
            }
            res.add(level);
        }
        return res;
    }

    // ===== 本地测试用, LeetCode 提交时删掉 =====
    public static void main(String[] args) {
        // Java 数组里用 null 标记空位(LeetCode 标配写法), 所以类型必须用 Integer 不能用 int
        Integer[] arr = {3, 9, 20, null, null, 15, 7};
        TreeNode root = buildTree(arr);
        System.out.println(new Solution().levelOrder(root));
    }

    // LeetCode 标准: 层序数组 → 二叉树(核心代码, 记下来)
    // 思路: 用 ArrayDeque 维护"待挂子节点的父节点", 下标 i 从 1 开始(根占 0 位).
    // 每弹一个父节点, 先挂左孩子(数组下一个非 null 位置), 再挂右孩子(再下一个).
    static TreeNode buildTree(Integer[] arr) {
        if (arr == null || arr.length == 0 || arr[0] == null) return null;
        TreeNode root = new TreeNode(arr[0]);
        ArrayDeque<TreeNode> q = new ArrayDeque<>();
        q.offer(root);
        int i = 1;
        while (!q.isEmpty() && i < arr.length) {
            TreeNode parent = q.poll();
            if (i < arr.length && arr[i] != null) {
                parent.left = new TreeNode(arr[i]);
                q.offer(parent.left);
            }
            i++;
        }
    }
}

```



```

        if (i < arr.length && arr[i] != null) {
            parent.right = new TreeNode(arr[i]);
            q.offer(parent.right);
        }
        i++;
    }
    return root;
}
}

```

C++ 对照

```

#include <iostream>
#include <vector>
#include <queue>
struct TreeNode {
    int val;
    TreeNode* left, * right;
    TreeNode(int v = 0) : val(v), left(nullptr), right(nullptr) {}
};

std::vector<std::vector<int>>> levelOrder(TreeNode* root) {
    std::vector<std::vector<int>>> res;
    if (!root) return res;
    std::queue<TreeNode*> q;
    q.push(root);
    while (!q.empty()) {
        int sz = q.size();
        std::vector<int> level;
        for (int i = 0; i < sz; i++) {
            TreeNode* node = q.front(); q.pop();
            level.push_back(node->val);
            if (node->left) q.push(node->left);
            if (node->right) q.push(node->right);
        }
        res.push_back(level);
    }
    return res;
}

```

重点记忆:buildTree(Integer[] arr) 这段层序数组解析代码是 LeetCode 标配,写一遍后存成模板,每次本地测试直接复用。

九、LeetCode 模式 I/O 简化(不用写 main)

9.1 类结构差异

项	ACM 模式(牛客)	LeetCode 模式
类名	public class Main	public class Solution
main	必须写	不写(平台帮你调)



异常声明	throws IOException	不写(平台用 try-catch 包好了)
输入	手动 BufferedReader 读	平台直接传参
输出	out.flush()	平台收集返回值

9.2 LeetCode 标准提交模板

```
class Solution {
    public 返回类型 方法名(参数类型 参数) {
        // 业务逻辑
        return 结果;
    }
}
```

9.3 自定义类的位置

如果题目要链表/树节点类,LeetCode 提交时把辅助类放在 Solution 同文件即可(类必须 static 修饰或顶层类,新版 LeetCode 可不加):

```
class ListNode {
    int val;
    ListNode next;
    ListNode(int v) { val = v; }
}

class Solution {
    public ListNode reverseList(ListNode head) {
        // ...
    }
}
```

9.4 本地测试套路(真用得上)

LeetCode 没法打断点调试,强烈建议在本地 main 里手写测试用例。把模板 5、模板 6 的 buildList / build Tree 辅助函数和 main 方法当成你的私人测试桩,改改 arr 就能验证。

十、性能对比(经验值,够用就行)

场景	慢方案	快方案	差距
100 万元素读取	Scanner(~3000 ms)	StreamTokenizer(~300 ms)	10 倍
100 万行输出	System.out.println 直接调(~3000 ms)	StringBuilder + PrintWriter(~200 ms)	15 倍
100 万次 nextLine	Scanner.nextLine(~2000 ms)	br.readLine(~200 ms)	10 倍

判断标准:

- 数据量 $\leq 10^4$:用谁都行,选可读性好的(Scanner / StringTokenizer)。
- 数据量 $\geq 10^5$:必须 BufferedReader + StreamTokenizer。
- 输出行数 $\geq 10^5$:必须 StringBuilder + PrintWriter。
- 牛客题面写"内存限制 64MB" 或 "时间限制 1s":直接上 StreamTokenizer,别赌。



十一、Java vs C++ I/O 对照总表

C++	Java	备注
<code>cin >> x</code>	<code>in.nextToken(); x = (int) in.nval;</code>	C++ 简洁Java 仪式感重
<code>getline(cin, s)</code>	<code>br.readLine()</code>	都读一整行
<code>cout << x << endl</code>	<code>out.println(x); out.flush();</code>	Java 必须显式 flush
<code>printf("%d", x)</code>	<code>String.format("%d", x)</code> 或 <code>out.printf</code>	Java 不常用
stringstream 切分	StringTokenizer	都是按空白切
<code>'\n'</code>	<code>'\n'</code> 或 <code>sb.append('\n')</code>	一致
<code>std::ios::sync_with_stdio(false)</code>	默认就快	Java 没这个开关

十二、章节小结(看完你得会)

必会项	重要度
throws IOException 怎么加	★★★
StreamTokenizer 三件套(nextToken / nval / sval)	★★★
PrintWriter 包装 BufferedWriter + flush	★★★
Scanner 换行符陷阱	★★
StringTokenizer 切一行	★★
<code>br.readLine() != null</code> 判 EOF	★
StreamTokenizer.TT_EOF 判 EOF	★
LeetCode 层序数组 → 二叉树 解析代码	★★

练习建议

- 把模板 1-4 自己手敲一遍,改 1-2 个变量名验证理解
- LeetCode 206 用模板 5 自己写一遍提交(顺便背下 ListNode 类)
- LeetCode 102 用模板 6 自己写一遍提交(顺便背下 TreeNode + buildTree 层序解析)
- 牛客"输入输出练习"专区挑 2-3 题用 v2 骨架实战(感受 ACM 模式完整流程)
- 找一道"输出数据量大的题"(比如牛客 HJ5 简单错误记录)对比 println 和 PrintWriter 的耗时差异

下一步

打开 02_语法差异速览.md,学 C++ 转 Java 必看的 7-8 个纯语法点(引用 / final / static / 类与对象 / char vs String / 异常处理最小化 / for-each),只讲刷题会碰到的,跳过工程向语法。

